

Phase 1: Entry Gateways & Access Lifecycles (Auth & Profile Domains)

Path A: The Onboarding Registration & Profile Initiation Loop

1. **Client Action:** The user submits registration data via `POST /api/v1/auth/register`.
2. **Gateway Screening:** The API Gateway identifies the request as an anonymous path, applying a rate-limiting check against the 50 requests/hour bucket. It forwards the request to the Authentication Service.
3. **Validation Inspection:** The Authentication Service checks input parameters against system rules:
 - *Failure Branch:* If the email fails regex validation, fields are missing, or the password fails security rules (minimum 6 characters, one uppercase, one number), the service breaks the flow, formatting a `VAL_WEAK_PASSWORD` or `VAL_REQUIRED_FIELD` error mapped into the standard JSON response envelope (`statusCode: 400`).
4. **Identity Generation & Persistence:**
 - The service runs a duplicate check against the `Users` database table. If the email is already registered, it aborts the process and returns an `AUTH_EMAIL_EXISTS` error (`statusCode: 409`).
 - *Success Branch:* If unique, the service cryptographically salts and hashes the plaintext password string, inserting an identity record into the `Users` table (`IsLockedOut = false`).
5. **Downstream Propagation:** The Authentication Service broadcasts a `user_registered` event to the message broker.
 - The **User Profile Service** consumes this event asynchronously, initializing a blank profile row in `UserProfiles` matching the new user identity key, setting `IsPremiumCached = false`, and setting up default layouts inside `UserPreferences` and `NotificationSettings`.
6. **Outbound Response:** The pipeline returns a `201 Created` payload to the client containing `requiresProfileCompletion: true`, prompt-locking the client interface into the onboarding screen.

Path B: The Login Validation & Security Enforcement Checkpoint

1. **Client Action:** The user submits their credentials via `POST /api/v1/auth/login`.
2. **Lockout Verification:** The Authentication Service queries the `Users` table and the `LoginAttempts` history log for that email address.

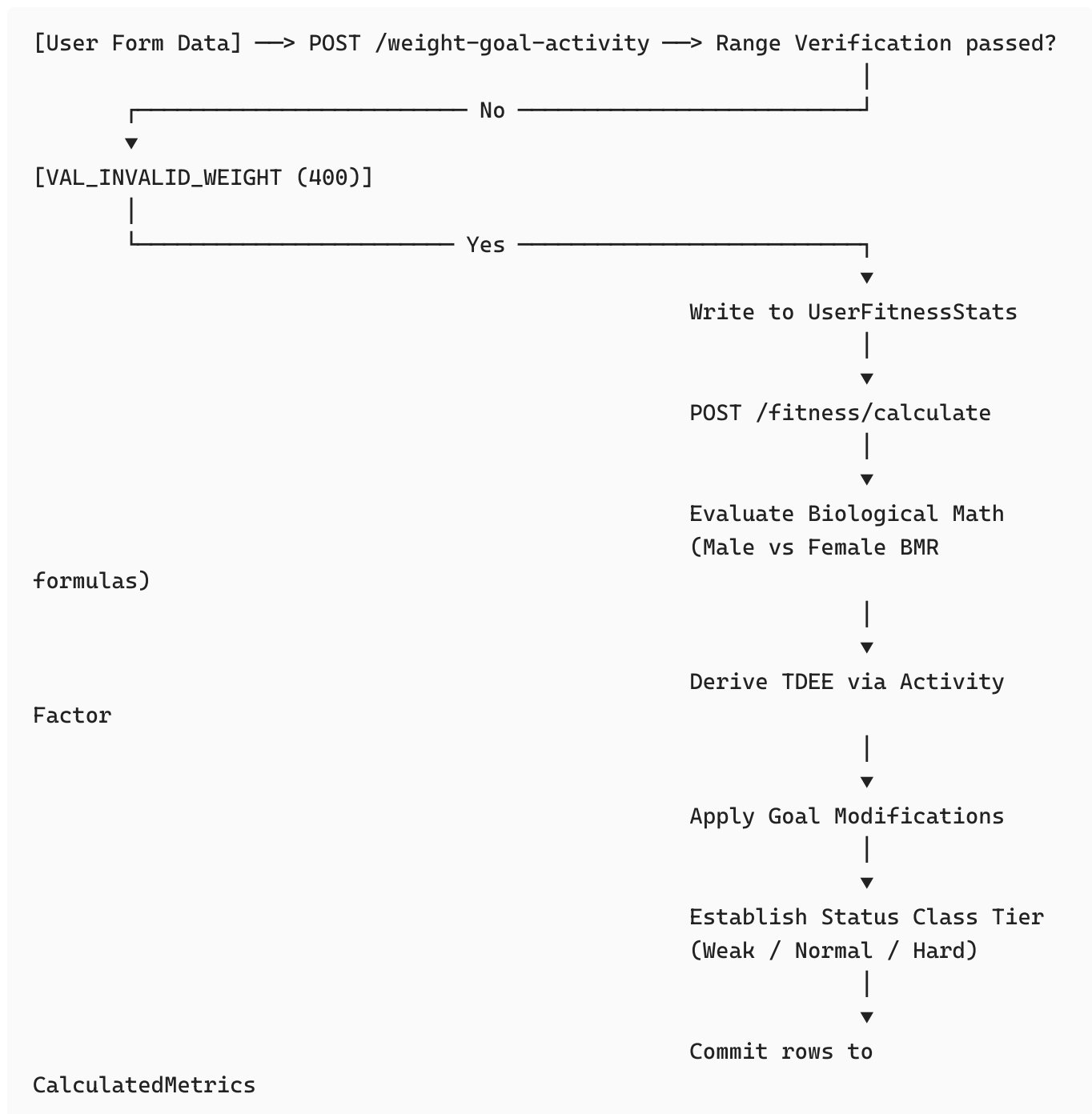
- *Lockout Failure Branch:* If `IsLockedOut` is evaluated as true and the current time is less than the `LockedUntil` value, the pipeline short-circuits instantly, returning an `AUTH_ACCOUNT_LOCKED` error (`statusCode: 423`).
3. **Password Assessment:** The engine runs a cryptographic match comparison of the submitted password against the `PasswordHash` entry in the `Users` table.
 - *Invalid Credentials Failure Branch:* If the password hash verification fails, the service writes a row into the `LoginAttempts` table marking `IsSuccess = false` alongside the client IP address. It checks the last 15 minutes of logs; if the failure count hits 5, it updates the user row to `IsLockedOut = true` and calculates `LockedUntil` as exactly 15 minutes into the future, returning an `AUTH_INVALID_CREDENTIALS` error (`statusCode: 401`).
 4. **Session Token Issuance:**
 - *Success Branch:* If the credentials match perfectly, the service drops any past failure counters, logs a successful attempt row in `LoginAttempts` , and issues a short-lived JSON Web Token (JWT) combined with a high-entropy `RefreshToken` committed to the database.
 - The JWT claims are signed with explicit tracking headers containing `X-User-Id` and `X-User-Tier: "Free"` (or `"Premium"` based on an internal look-up check).
 5. **Client Access Elevation:** The gateway intercepts the outbound payload, reading the token claims to configure local cache buckets, and outputs a `200 OK` response. The client state is updated to grant entry to the dashboard.

Path C: The Credential Recovery (OTP Failover) Loop

1. **Client Action:** A user clicks "Forgot Password" and submits an email via `POST /api/v1/auth/forgot-password` .
2. **Challenge Generation:** The Authentication Service verifies the email's existence. If valid, it generates a cryptographically random, non-predictable 6-digit challenge token, writes its hash signature to the `OtpCodes` table (`ExpiresAt = +10 minutes` , `IsUsed = false`), and passes the code to mail queues.
 - *Throttle Check:* A Redis key restricts subsequent generation calls on this account for 30 seconds. If violated, it drops the thread with a `RATE_OTP_RESEND_TOO_SOON` error (`statusCode: 429`).
3. **Verification Exchange:** The user inputs the code via `POST /api/v1/auth/verify-otp` . The engine checks the entry against the `OtpCodes` record.
 - *Failure Branch:* If the input does not match, or `ExpiresAt` has passed, or `IsUsed == true` , it rejects the submission returning `AUTH_INVALID_OTP` or `AUTH_OTP_EXPIRED` (`statusCode: 400`).
 - *Success Branch:* If it matches, the service flips `IsUsed = true` and outputs a temporary, high-entropy, short-lived `resetToken` .

4. **Execution Override:** The user submits their new credentials via `POST /api/v1/auth/reset-password`. The engine confirms that `newPassword` and `confirmPassword` are identical, updates the target account `PasswordHash` inside the `Users` table, and marks all active user rows in `RefreshTokens` as revoked to force immediate re-authentication across all active devices.

Phase 2: Core Biometrics Ingestion & Metabolic Blueprint Generation (FCE Core Loop)



1. **Client Action:** The user inputs physical parameters via `POST /api/v1/fitness/weight-goal-activity`.

2. **Biometric Validation:** The **Fitness Calculation Engine (FCE)** runs an internal validation pass over incoming attributes:
 - *Failure Branch:* If age is outside 16–100, weight outside 40–200 kg, or height outside 140–220 cm, the process halts, returning an explicit biometric error code (`VAL_INVALID_WEIGHT`, `VAL_INVALID_HEIGHT`, etc.) with a `400 Bad Request` status.
 - *Success Branch:* If all checks pass, data is committed to the `UserFitnessStats` database table under the user's primary identity key.
3. **Metabolic Computation Execution:** The client invokes `POST /api/v1/fitness/calculate`. The FCE extracts the user's `UserFitnessStats` record and executes standard biological equations:
 - **Male BMR Formula:** $BMR = (10 \times \text{weight}_{\text{kg}}) + (6.25 \times \text{height}_{\text{cm}}) - (5 \times \text{age}_{\text{years}}) + 5$
 - **Female BMR Formula:** $BMR = (10 \times \text{weight}_{\text{kg}}) + (6.25 \times \text{height}_{\text{cm}}) - (5 \times \text{age}_{\text{years}}) - 161$
4. **Energy Expenditure Derivation:** The computed BMR is multiplied by the user's selected `ActivityLevel` factor to establish their Total Daily Energy Expenditure (TDEE): `Rookie` (1.2), `Beginner` (1.375), `Intermediate` (1.55), `Advance` (1.725), or `TrueBeast` (1.9).
5. **Macro Alignment Calibration:** The system calculates the definitive daily `CalorieTarget` by applying objective-oriented offsets to the TDEE:
 - `Lose Weight` → Deduct 500 calories ($TDEE - 500$).
 - `Gain Weight` → Add 300 calories ($TDEE + 300$).
 - `Get Fitter` → Maintain base TDEE ($TDEE$).
6. **Difficulty Tiering Classification:** The resulting calorie target establishes the user's system classification tier: `Weak` (≤ 1800 calories), `Normal` (1801 to 2500 calories), or `Hard` (> 2500 calories). The output variables are saved into the `CalculatedMetrics` data table.
7. **Automated Plan Assignment:** The system invokes `POST /api/v1/fitness/assign-plan`. The engine references the `FitnessPlanConfigs` rule matrix to match the configuration where the user's `Goal` and calculated calorie `Status` intersect.
 - *History Trail Logging:* Before updating the active plan, the system checks for an existing assignment row in `UserAssignedPlans`. If found, it flips `IsActive = false`, copies the snapshot details into the immutable `UserPlanHistory` log (storing `AssignedAt`, `EndedAt`, and the transitional `ReasonForChange`), and writes the new plan routing record to `UserAssignedPlans`.

Phase 3: The Active Training Lifecycle & Performance Analytics (Workout & Progress Loop)

1. Catalog Exploration and Routine Interception

- The user requests their schedule via `GET /api/v1/workouts?category=chest&difficulty=intermediate`.

- The **Workout Service** executes an inner join across `Workouts` , `WorkoutExercises` , and `Exercises` to return a structured response payload. It automatically sorts the workout checklist sequence by the configuration tracking value `OrderIndex ASC`.

2. Session Initialization Step

- The user hits "Start Workout," calling `POST /api/v1/workouts/{id}/start` .
- The Workout Service inserts an active tracking row instance into the `WorkoutSessions` table with an initial status of `"Active"` , returning a unique `SessionId` token used to bind real-time execution logs.

3. Granular Performance Logging Loop

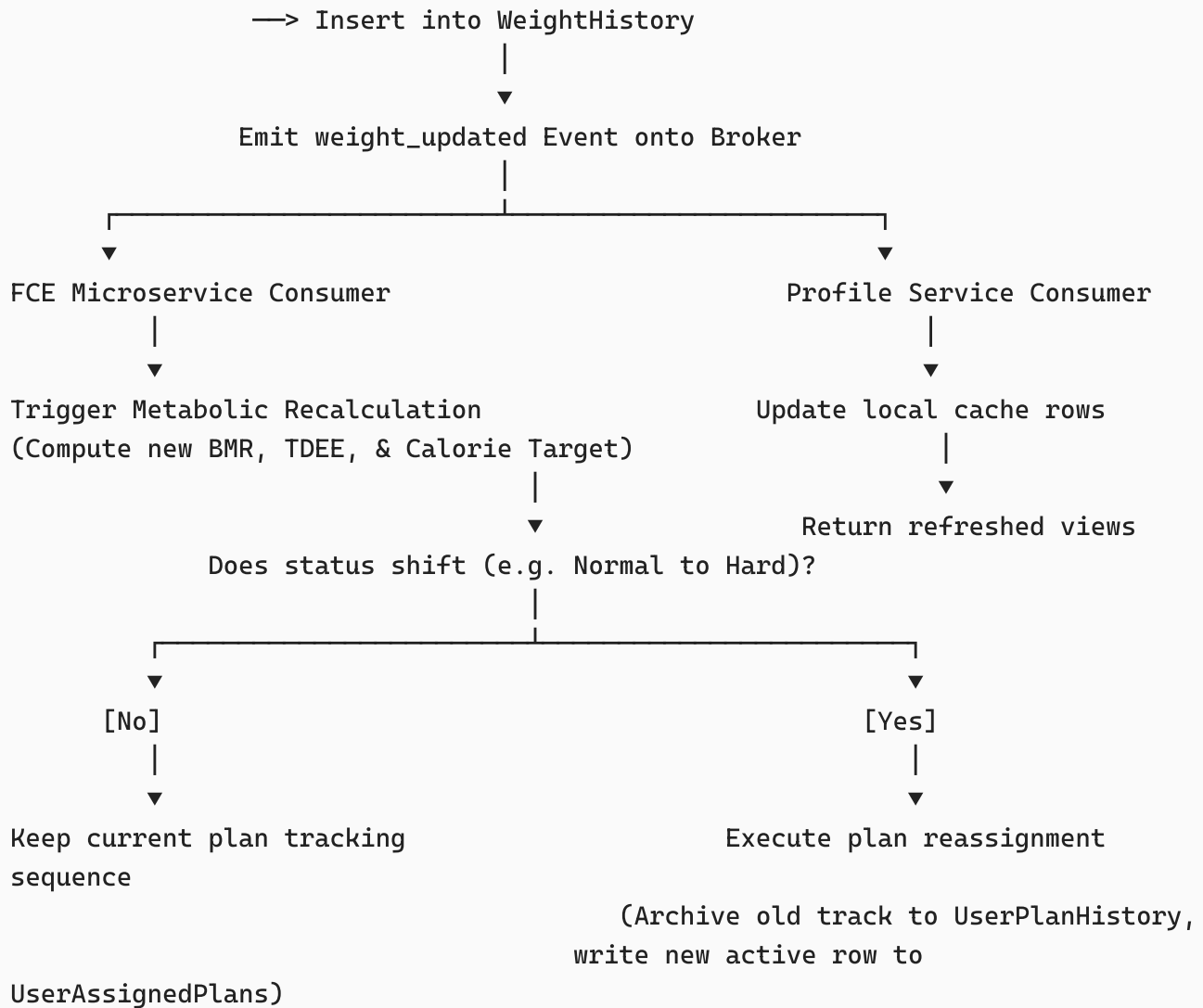
- Upon completing the routine, the client transmits the performance dataset via `POST /api/v1/progress/workouts` .
- **Transactional Integrity Pass:** The **Progress Tracking Service** executes an atomic transaction block:
 - It updates the active tracking row state within `WorkoutSessions` from `"Active"` to `"Completed"` , recording the total time spent and calories burned.
 - It writes a master summary row into `WorkoutLogs` and loops through the array payload to insert set-by-set child records (`SetsCompleted` , `RepsCompleted` , `WeightUsed`) directly into `WorkoutLogExercises` .
 - *Failure Safeguard:* If the user's workout rating score falls outside the allowable 1–5 range, or if the `SessionId` doesn't match an active tracking record, the entire transaction rolls back immediately, returning a `RES_SESSION_NOT_FOUND` error code (`statusCode: 404`).

4. Metrics Aggregation & Consistency Evaluation

- Post-completion, the Progress Tracking Service evaluates training consistency:
 - It checks the user's `Streaks` entry row, comparing the current workout timestamp with the `LastWorkoutDate` . If the gap is exactly 1 calendar day, it increments the `CurrentStreak` counter by 1. If the gap spans the same calendar day, the counter is preserved. If the gap exceeds 24 hours, the streak is reset to 1.
 - If `CurrentStreak` exceeds the `LongestStreak` high-water mark, it updates `LongestStreak` to match.
 - Global values inside `UserStatistics` (`TotalWorkouts` incremented by 1, `TotalCaloriesBurned` accumulated) are recalculated and committed to storage.

5. Biometric Variance Tracking & Recalculation Triggers (The Async Catalyst)

[User Updates Weight] → POST /progress/weight



1. **Client Action:** The user logs their morning weight via POST /api/v1/progress/weight .
2. **Local Logging:** The Progress Tracking Service validates that the weight falls within the allowable 40–200 kg boundary, appends an entry to the WeightHistory table, and modifies the CurrentWeight field inside the local UserStatistics summary repository.
3. **Event Emitting:** Upon a successful database transaction write loop, the service broadcasts a weight_updated event package containing UserId , NewWeight , and a tracking timestamp onto the message broker bus.
4. **Asynchronous Recalculation Ingestion:** The **FCE Microservice** consumes the weight_updated event message from the queue, bypasses synchronous HTTP request pipelines, and instantly triggers its calculation formulas using the new weight metrics:
 - It derives new BMR, TDEE, and daily calorie target scores.
 - If the recalculated calorie target causes the user's difficulty classification status to shift (e.g., from Normal to Hard), it triggers an automated plan reassignment sequence.

- The current plan assignment row is archived to `UserPlanHistory` , and a new active routine tracking row matching the updated difficulty parameters is written to `UserAssignedPlans` .

Phase 4: Secondary Domain Feature Operations (Nutrition & Smart Coach Lookup Engine)

Path A: Targeted Dietary Matching

1. **Client Action:** The user navigates to the diet tab, invoking `GET /api/v1/nutrition/recommendations/{userId}` .
2. **Inter-Service Lookup:** The **Nutrition Service** captures the request. To ensure it respects the user's latest metabolic state, it makes a synchronous internal HTTP/REST call back to the FCE Service to read the live computed `CalorieTarget` and classification `Status` from the `CalculatedMetrics` repository.
3. **Query Execution & Filtering:** Armed with the calorie constraints, the Nutrition Service queries its database. It filters rows out of the `Meals` table where individual meal calories are within safe range bounds of the user's target profile, sorting the recipe structures into breakfast, lunch, and dinner arrays.
4. **Payload Composition:** The system merges the user's daily goals alongside matching meal records, outputting a detailed nutritional blueprint conforming to the global envelope layout.

Path B: AI Coach Conversation & Landing Feed Aggregation

1. **Client Action:** The user opens a chat thread and types a message, calling `POST /api/v1/smart-coach/chat` .
2. **Context Enrichment & AI Ingestion:** The **Smart Coach Service** logs the incoming prompt string directly into `ChatMessages` linked to an active `SessionId` .
 - Before forwarding the question to backend AI models, it performs rapid parallel synchronous read operations downstream: it pulls current biological goals from the FCE and live activity trends (streaks, total workouts completed) from the Progress Tracking service.
 - It injects these metrics as a system prompt wrapper so the AI outputs hyper-personalized advice rather than generic recommendations.
3. **Dashboard Landing Feed Optimization:** The user navigates back to the home screen dashboard, triggering `GET /api/v1/home` .
 - **High-Speed Cache Check:** The service intercepts the routing path and scans the local `RecommendationCache` table for the requesting `UserId` .
 - *Cache Hit Path:* If a record exists and `DateTime.UtcNow < ExpiresAt` , the service skips all downstream network queries. It serializes and returns the cached JSON data

block directly to the client under a 50ms response timeframe.

- *Cache Miss/Expiration Path:* If the cache row has expired, the orchestrator executes parallel async lookups across the entire architecture topology (pulling user details from Profiles, calorie targets from FCE, upcoming routines from Workouts, meal recommendations from Nutrition, and stats from Progress)[cite: 2]. It aggregates the distinct payloads into a unified dashboard view, writes the composite JSON block back to `RecommendationCache` (setting a 30-minute sliding expiration window), and returns the data to the client.

Phase 5: Monetization & System Notification Infrastructure (Billing & Alert Dispatches)

Path A: Payment Gateway Mocking & Tier Transition Escalations

1. **Client Action:** A free-tier user upgrades their account tier by invoking `POST /api/v1/subscription/upgrade` [cite: 2].
2. **Financial Logging:** The **Subscription & Billing Service** intercepts the call, validates the transaction using a simulated check out loop, inserts a financial ledger entry line into `BillingLogs`, and updates the user's record in `UserSubscriptions` to `Tier = 'Premium'`, `Status = 'Active'`, setting `ExpiresAt` to exactly 30 days out.
3. **High-Priority Architecture Propagation:** The service publishes a high-priority integration event named `subscription_upgraded` across the broker exchange bus.
4. **Downstream Cache Synchronization:**
 - **User Profile Service Integration:** The Profile Service consumes the event and instantly updates its database row setting `IsPremiumCached = true` inside `UserProfiles` [cite: 2].
 - **API Gateway Enforcement:** The API Gateway intercepts the event notification and invalidates its local distributed memory caches. The user's rate-limiting boundaries are dynamically elevated from the standard 500 requests/hour limit to the unthrottled 2000 requests/hour premium tier allocation.
5. **Soft-Capped Gate Clearance:** When the user initiates subsequent complex operations (such as unrestricted Smart Coach prompt requests or advanced metric outputs), security check layers query the updated local caches, confirming premium access rights and instantly granting unthrottled permission paths.

Path B: Asynchronous Milestone Reward Notification Dispatches

1. **Background Event Trigger:** While processing a workout log inside the Progress Tracking Service, the counter evaluation engine notes that a user has successfully unlocked a target milestone (e.g., completing a 7-day training consistency stretch)[cite: 2].

2. **Award Registration:** The Progress Service inserts a milestone record row directly into `UserAchievements` [cite: 2], maps the reward icon relation out of the master `Achievements` table [cite: 2], and emits an asynchronous integration notification event (`achievement_earned`) onto the system message queue.
 3. **Notification Feed Ingestion:** The **Notification Service** daemon background worker consumes the `achievement_earned` message from the queue, formats a localized message text layout, and appends an unread row entry directly into the `InAppNotifications` database table (`IsRead = false`).
 4. **Feed Invalidation:** The client application interface registers the background alert update, prompting the user to invoke `GET /api/v1/notifications` to render their unread alerts feed. The user taps the notification, calling `PUT /api/v1/notifications/{id}/read`, which toggles the `IsRead` boolean field to true inside database tables, clearing out client badges cleanly.
-
-
-
-
-

Flow 1: User Registration & Profile Initialization

[User Form Data]

- > 1. Client invokes `POST /api/v1/auth/register`
- > 2. API Gateway intercepts request —> Validates anonymous rate limit bucket (50 req/hr max)
- > 3. Auth Service processes payload —> Executes validation regex checks (Email format & strict password complexity)
- > 4. Auth Service queries `Users` table —> [If Email exists] —> Abort —> Return `AUTH_EMAIL_EXISTS (409)`
- > 5. Auth Service runs cryptographic salt/hash on password string (e.g., `BCrypt/Argon2id`)
- > 6. Auth Service executes database mutation: `INSERT INTO Users (Email, PasswordHash, IsLockedOut, CreatedAt)`

—> 7. Auth Service broadcasts integration event "user_registered" (UserId, Email) onto the Message Broker Bus

—> 8. Auth Service returns 201 Created response envelope to Client —> Sets requiresProfileCompletion: true

—> 9. User Profile Service background consumer pulls "user_registered" event from Message Broker

—> 10. Profile Service executes database mutation: INSERT INTO UserProfiles (Id, Email, IsPremiumCached) VALUES (@UserId, @Email, false)

—> 11. Profile Service executes database mutation: INSERT INTO UserPreferences & NotificationSettings (Default configurations)

—> 12. Client UI switches to onboarding lock screen state

Flow 2: User Login & Brute-Force Lockout Protection

[User Credentials]

—> 1. Client invokes POST /api/v1/auth/login

—> 2. Auth Service reads target row from Users table matching Email parameter

—> 3. Auth Service evaluates Users.IsLockedOut status flag

—> 4. [Branch: Active Lockout] —> If IsLockedOut == true AND DateTime.UtcNow < LockedUntil —> Abort —> Return AUTH_ACCOUNT_LOCKED (423)

—> 5. [Branch: Evaluation Pass] —> Auth Service asserts cryptographically computed input hash against stored PasswordHash

—> 6. [Sub-Branch: Password Mismatch]

—> 6a. Auth Service executes database mutation: INSERT INTO LoginAttempts (Email, AttemptedAt, IsSuccess = false, IPAddress)

—> 6b. Auth Service queries last 15 minutes of failure rows for this Email

—> 6c. If Failure Count == 5 —> UPDATE Users SET IsLockedOut = true, LockedUntil = (DateTime.UtcNow + 15 minutes)

—> 6d. Return AUTH_INVALID_CREDENTIALS (401)

—> 7. [Sub-Branch: Password Success]

—> 7a. Auth Service executes database mutation: INSERT INTO LoginAttempts (Email, AttemptedAt, IsSuccess = true, IPAddress)

—> 7b. Auth Service issues access JWT claims payload containing custom headers (X-User-Id, X-User-Tier: "Free")

—> 7c. Auth Service generates secure long-lived rotation token

—> INSERT INTO RefreshTokens (UserId, Token, ExpiresAt)

—> 7d. Return 200 OK with signed JWT token, RefreshToken, profileCompleted, and isPremium flags to Client

Flow 3: Biometric Data Ingestion & Metabolic Plan Assignment (FCE Engine)

[Biometric Ingestion Form]

—> 1. Client invokes POST /api/v1/fitness/weight-goal-activity

—> 2. FCE Service checks validation boundaries —> Age (16-100), Weight (40-200 kg), Height (140-220 cm), Gender/Goal enums

—> 3. [Validation Failure] —> If checks fail —> Abort —> Return explicit validation error (e.g., VAL_INVALID_WEIGHT) (400)

—> 4. [Validation Pass] —> FCE executes database mutation: INSERT INTO UserFitnessStats (UserId, Weight, Height, Age, Gender, Goal, ActivityLevel)

—> 5. Client invokes computational transition endpoint: POST /api/v1/fitness/calculate

—> 6. FCE processes math formulas based on biological Gender flag:

—> Male BMR = $(10 * \text{Weight}) + (6.25 * \text{Height}) - (5 * \text{Age}) + 5$

—> Female BMR = $(10 * \text{Weight}) + (6.25 * \text{Height}) - (5 * \text{Age}) - 161$

—> 7. FCE calculates Total Daily Energy Expenditure: $\text{TDEE} = \text{BMR} * \text{ActivityLevel Multiplier}$ (Rookie=1.2 up to TrueBeast=1.9)

—> 8. FCE derives final CalorieTarget based on User Fitness Goal:

—> Lose Weight = $\text{TDEE} - 500$ | Gain Weight = $\text{TDEE} + 300$ | Get Fitter = TDEE

—> 9. FCE evaluates Calorie Target classification status tier:

—> Target ≤ 1800 kcal = "Weak" | 1801 to 2500 kcal = "Normal"

```
| > 2500 kcal = "Hard"
```

—> 10. FCE executes database mutation: UPSERT INTO CalculatedMetrics (UserId, Bmr, Tdee, CalorieTarget, Status, CalculatedAt)

—> 11. FCE handles automated allocation pass: System invokes POST /api/v1/fitness/assign-plan

—> 12. FCE queries master matrix rules config table: SELECT ExternalPlanId FROM FitnessPlanConfigs WHERE Goal = @Goal AND Status = @Status

—> 13. FCE scans existing rows inside active tracking structures: SELECT * FROM UserAssignedPlans WHERE UserId = @UserId AND IsActive = true

—> 14. [History Logging Path] —> If existing active plan row found:

—> 14a. FCE executes mutation: UPDATE UserAssignedPlans SET IsActive = false WHERE UserId = @UserId

—> 14b. FCE executes mutation: INSERT INTO UserPlanHistory (UserId, ExternalPlanId, AssignedAt, EndedAt, ReasonForChange = "initial_assignment")

—> 15. FCE executes database mutation: INSERT INTO UserAssignedPlans (UserId, ExternalPlanId, AssignedAt, IsActive = true)

—> 16. Return 200 OK response containing newly mapped Plan allocation variables to client UI

Flow 4: Workout Session Execution & Set-Volume Logging

[User Opens Routine Catalog]

—> 1. Client pulls routine details: GET /api/v1/workouts/{id}

—> 2. Workout Service executes an inner join index query across tables: Workouts —> WorkoutExercises —> Exercises

—> 3. Workout Service sorts compilation array sequentially by template index parameter: ORDER BY OrderIndex ASC

—> 4. User presses "Start" —> Client invokes transition tracker initializer: POST /api/v1/workouts/{id}/start

—> 5. Workout Service executes database mutation: INSERT INTO WorkoutSessions (SessionId, UserId, WorkoutId, StartedAt, Status = "Active")

—> 6. Workout Service returns unique session string tracking code (SessionId) along with ordered exercise template checklist to Client

—> 7. User completes workout —> Client transmits final performance dataset array via POST /api/v1/progress/workouts

—> 8. Progress Tracking Service receives payload and starts a database Atomic Transaction Block

—> 9. Progress Service validates constraints —> Verifies SessionId matches an active record & checks workout Rating (must be 1-5)

—> 10. Progress Service calls Workout Service synchronously via REST —> UPDATE WorkoutSessions SET Status = "Completed" WHERE SessionId = @SessionId

—> 11. Progress Service executes master mutation: INSERT INTO WorkoutLogs (UserId, WorkoutId, SessionId, DurationInMinutes, CaloriesBurned, Rating, Notes, CompletedAt)

—> 12. Progress Service loops nested arrays and executes multi-row mutation: INSERT INTO WorkoutLogExercises (WorkoutLogId, ExerciseId, SetsCompleted, RepsCompleted, WeightUsed, Completed)

—> 13. Progress Service updates global metric counter row variables: UPDATE UserStatistics SET TotalWorkouts = TotalWorkouts + 1, TotalCaloriesBurned = TotalCaloriesBurned + @Burned WHERE Id = @UserId

—> 14. Progress Service extracts Streaks tracker record row: SELECT LastWorkoutDate, CurrentStreak, LongestStreak FROM Streaks WHERE UserId = @UserId

—> 15. Progress Service evaluates calendar date gaps:

—> [If Gap == 1 Day] —> UPDATE Streaks SET CurrentStreak = CurrentStreak + 1, LastWorkoutDate = UTC_NOW

—> [If Gap > 1 Day] —> UPDATE Streaks SET CurrentStreak = 1, LastWorkoutDate = UTC_NOW

—> 16. Progress Service checks streak high-water marks —> If CurrentStreak > LongestStreak —> UPDATE Streaks SET LongestStreak = CurrentStreak

—> 17. Database Transaction Block commits successfully —> Return 201 Created unified envelope response to Client

Flow 5: Weight Variance Check-in & Asynchronous Plan Recalculation Loop

[User Checks-In Weight]

—> 1. Client invokes biometric update endpoint: POST
/api/v1/progress/weight

—> 2. Progress Service checks validation —> Weight parameter must be between 40kg and 200kg boundary thresholds

—> 3. Progress Service executes local database mutation: INSERT INTO WeightHistory (UserId, Weight, Date, Notes)

—> 4. Progress Service updates local summary cache row table: UPDATE UserStatistics SET CurrentWeight = @NewWeight WHERE Id = @UserId

—> 5. Progress Service broadcasts asynchronous integration event to Message Broker: Exchange: "weight-events" | RoutingKey: "weight.updated" | Payload: { UserId, NewWeight, RecordedAt }

—> 6. Progress Service instantly drops its synchronous processing thread
—> Returns 201 Created progress snapshot variables to Client UI

—> 7. FCE Service background message queue daemon worker consumes the "weight_updated" event from the Message Broker Bus

—> 8. FCE background worker extracts historical biometrics rows: SELECT * FROM UserFitnessStats WHERE UserId = @UserId

—> 9. FCE background worker re-calculates metabolic formulas using the updated weight value (Computes fresh BMR, TDEE, and CalorieTarget)

—> 10. FCE background worker assesses whether the newly computed calorie target changes the system status tier classification

—> 11. [Branch: Status Tier Shifts (e.g., Normal to Hard)]

—> 11a. FCE updates old tracking plan row assignment: UPDATE UserAssignedPlans SET IsActive = false WHERE UserId = @UserId

—> 11b. FCE logs history trail row: INSERT INTO UserPlanHistory (UserId, ExternalPlanId, AssignedAt, EndedAt, ReasonForChange = "weight_update")

—> 11c. FCE queries plan configuration matrix and applies new

```
active plan: INSERT INTO UserAssignedPlans (UserId, ExternalPlanId,
AssignedAt, IsActive = true)
```

—> 12. [Branch: Status Tier Unchanged] —> FCE executes mutation: UPDATE CalculatedMetrics SET CalorieTarget = @NewTarget, Bmr = @NewBmr, Tdee = @NewTdee WHERE UserId = @UserId

Flow 6: Premium Membership Upgrade & Gateway Rate Limit Modification

[User Clicks Membership Purchase]

—> 1. Client invokes transaction checkout route: POST /api/v1/subscription/upgrade

—> 2. Subscription & Billing Service processes simulated gateway checks —> Generates a secure ReferenceId receipt token string

—> 3. Billing Service executes database audit log mutation: INSERT INTO BillingLogs (UserId, ReferenceId, Amount, Currency, PaymentStatus = "Success", Timestamp)

—> 4. Billing Service updates billing profile row: UPSERT INTO UserSubscriptions (UserId, Tier = "Premium", Status = "Active", StartsAt = UTC_NOW, ExpiresAt = UTC_NOW + 30 days)

—> 5. Billing Service broadcasts high-priority integration event to Message Broker Bus: "subscription_upgraded" (UserId, Tier = "Premium", ExpiresAt)

—> 6. Billing Service returns 200 OK premium state payload envelope —> Client unlocks advanced UI features

—> 7. User Profile Service event consumer pulls "subscription_upgraded" event from Message Broker —> Executes database mutation: UPDATE UserProfiles SET IsPremiumCached = true WHERE Id = @UserId

—> 8. API Gateway core proxy reverse caching layer consumes the "subscription_upgraded" event message background alert

—> 9. API Gateway invalidates its distributed memory cache claims bucket for the target user's active token identity

—> 10. Gateway shifts user traffic permissions check rules automatically

—> Moves user validation claims parameters from Authenticated Free (500 req/hr) up to the Premium Access Tier (2000 req/hr) dynamic ceiling window